Linear Discriminant Analysis (LDA)

Lieven Clement

Invalid Date

Table of contents

1 Breast cancer example **1**
1.1 Data . . . 2

2 Linear discriminant analysis **3**
2.1 Between and within sums of squares . . . 3
2.2 Obtain projections . . . 5
2.3 More than two classes . . . 6

3 High dimensional predictors **6**

4 Breast cancer example **7**
4.1 All genes . . . 7
4.1.1 LDA . . . 7
4.1.2 Interpretation of loadings . . . 9
4.2 Sparse LDA based on 150 random genes . . . 10
4.3 LDA based on 150 random genes . . . 13
4.4 Wrapup . . . 15

Acknowledgement **15**

Session info **16**

```
library(tidyverse)
library(gridExtra)
```

1 Breast cancer example

- Schmidt *et al.*, 2008, Cancer Research, **68**, 5405-5413

- Gene expression patterns in n=200 breast tumors were investigated (p=22283 genes)
- After surgery the tumors were graded by a pathologist (stage 1,2,3)

1.1 Data

```
#BiocManager::install("genefu")
#BiocManager::install("breastCancerMAINZ")

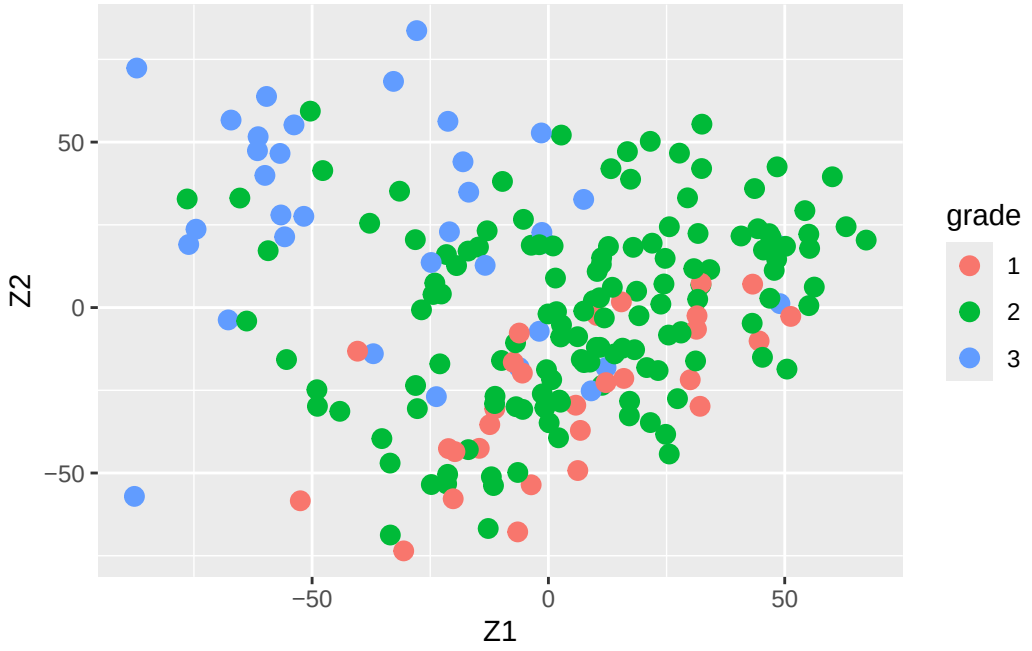
library(genefu)
library(breastCancerMAINZ)
data(mainz)

X <- t(exprs(mainz)) # gene expressions
n <- nrow(X)
H <- diag(n)-1/n*matrix(1,ncol=n,nrow=n)
X <- H%*%X
Y <- pData(mainz)$grade
table(Y)
```

```
#> Y
#>   1   2   3
#>  29 136  35
```

```
svdX <- svd(X)
k <- 2
Zk <- svdX$u[,1:k] %*% diag(svdX$d[1:k])
colnames(Zk) <- paste0("Z",1:k)

Zk |>
  as.data.frame() |>
  mutate(grade = Y |> as.factor()) |>
  ggplot(aes(x= Z1, y = Z2, color = grade)) +
  geom_point(size = 3)
```



2 Linear discriminant analysis

Fisher's construction of LDA is simple: it allows for classification in a dimension-reduced subspace of $\mathbb{R}^p$.

First we assume that Y can only take two values (0/1).

Fisher aimed for a direction, say $\mathbf{a}$, in the p-dimensional predictor space such that the orthogonal projections of the predictors, $\mathbf{x}^t\mathbf{a}$, show maximal ratio between the between and within sums of squares:

$$\mathbf{v} = \text{ArgMax}_a \frac{\mathbf{a}^t\mathbf{B}\mathbf{a}}{\mathbf{a}^t\mathbf{W}\mathbf{a}} \text{ subject to } \mathbf{a}^t\mathbf{W}\mathbf{a} = 1,$$

where $\mathbf{W}$ and $\mathbf{B}$ are the within and between covariance matrices of $\mathbf{x}$. The restriction is introduced to obtain a (convenient) unique solution.

2.1 Between and within sums of squares

- In the training dataset, let $\mathbf{x}_{ik}$ denote the ith p-dimensional observation in the kth group ($k = 0, 1$ referring to $Y = 0$ and $Y = 1$, resp.), $i = 1, \ldots, n_k$.

- Let $z_{ik} = \mathbf{a}^t \mathbf{x}_{ik}$ denote the orthogonal projection of $\mathbf{x}_{ik}$ onto $\mathbf{a}$
- For the one-dimensional z-observations, consider the following sum of squares:

$$\text{SSE} = \text{within sum of squares} = \sum_{k=0,1} \sum_{i=1}^{n_k} (z_{ik} - \bar{z}_k)^2$$

$$\text{SSB} = \text{between sum of squares} = \sum_{k=0,1} \sum_{i=1}^{n_k} (\bar{z}_k - \bar{z})^2 = \sum_{k=0,1} n_k (\bar{z}_k - \bar{z})^2$$

  with $\bar{z}_k$ the sample mean of z_{ik} within group k, and $\bar{z}$ the sample mean of all z_{ik}.
- To reformulate SSE and SSB in terms of the p-dimensional $\mathbf{x}_{ik}$, we need the sample means

$$\bar{z}_k = \frac{1}{n_k} \sum_{i=1}^{n_k} z_{ik} = \frac{1}{n_k} \sum_{i=1}^{n_k} \mathbf{a}^t \mathbf{x}_{ik} = \mathbf{a}^t \frac{1}{n_k} \sum_{i=1}^{n_k} \mathbf{x}_{ik} = \mathbf{a}^t \bar{\mathbf{x}}_k$$

$$\bar{z} = \frac{1}{n} \sum_{k=0,1} \sum_{i=1}^{n_k} z_{ik} = \cdots = \mathbf{a}^t \bar{\mathbf{x}}.$$

- SSE becomes

$$\text{SSE} = \sum_{k=0,1} \sum_{i=1}^{n_k} (z_{ik} - \bar{z}_k)^2 = \mathbf{a}^t \left(\sum_{k=0,1} \sum_{i=1}^{n_k} (\mathbf{x}_{ik} - \bar{\mathbf{x}}_k)(\mathbf{x}_{ik} - \bar{\mathbf{x}}_k)^t \right) \mathbf{a}$$

- SSB becomes

$$\text{SSB} = \sum_{k=0,1} n_k (\bar{z}_k - \bar{z})^2 = \mathbf{a}^t \left(\sum_{k=0,1} n_k (\bar{\mathbf{x}_k} - \bar{\mathbf{x}})(\bar{\mathbf{x}_k} - \bar{\mathbf{x}})^t \right) \mathbf{a}$$

- The $p \times p$ matrix

$$\mathbf{W} = \sum_{k=0,1} \sum_{i=1}^{n_k} (\mathbf{x}_{ik} - \bar{\mathbf{x}}_k)(\mathbf{x}_{ik} - \bar{\mathbf{x}}_k)^t$$

  is referred to as the matrix of **within** sum of squares and cross products.
- The $p \times p$ matrix

$$\mathbf{B} = \sum_{k=0,1} n_k (\bar{\mathbf{x}_k} - \bar{\mathbf{x}})(\bar{\mathbf{x}_k} - \bar{\mathbf{x}})^t$$

  is referred to as the matrix of **between** sum of squares and cross products.
- Note that on the diagonal of $\mathbf{W}$ and $\mathbf{B}$ you find the ordinary univariate within and between sums of squares of the individual components of $\mathbf{x}$.

2.2 Obtain projections

An equivalent formulation:

$$\mathbf{v} = \text{ArgMax}_a \mathbf{a}^t\mathbf{B}\mathbf{a} \text{ subject to } \mathbf{a}^t\mathbf{W}\mathbf{a} = 1.$$

This can be solved by introducing a Lagrange multiplier:

$$\mathbf{v} = \text{ArgMax}_a \mathbf{a}^t\mathbf{B}\mathbf{a} - \lambda(\mathbf{a}^t\mathbf{W}\mathbf{a} - 1).$$

Calculating the partial derivative w.r.t. $\mathbf{a}$ and setting it to zero gives

$$\begin{aligned} 2\mathbf{B}\mathbf{a} - 2\lambda\mathbf{W}\mathbf{a} &= 0 \\ \mathbf{B}\mathbf{a} &= \lambda\mathbf{W}\mathbf{a} \\ \mathbf{W}^{-1}\mathbf{B}\mathbf{a} &= \lambda\mathbf{a}. \end{aligned}$$

From the final equation we recognise that $\mathbf{v} = \mathbf{a}$ is an **eigenvector** of $\mathbf{W}^{-1}\mathbf{B}$, and λ is the corresponding **eigenvalue**.

The equation has in general rank($\mathbf{W}^{-1}\mathbf{B}$) solutions. In the case of two classes, the rank equals 1 and thus only one solution exists.

- A training data set is used for the calculation of $\mathbf{W}$ and $\mathbf{B}$. $\longrightarrow$ This gives the eigenvector $\mathbf{v}$
- The training data is also used for the calculation of the centroids of the classes (e.g. the sample means, say $\bar{\mathbf{x}}_1$ and $\bar{\mathbf{x}}_2$). $\longrightarrow$ The projected centroids are given by $\bar{\mathbf{x}}_1^t\mathbf{v}$ and $\bar{\mathbf{x}}_2^t\mathbf{v}$.
- A new observation with predictor $\mathbf{x}$ is classified in the class for which the projected centroid is closest to the projected predictor $z = \mathbf{x}^t\mathbf{v}$.

An advantage of this approach is that $\mathbf{v}$ can be interpreted (similar as the loadings in a PCA) in terms of which predictors x_j are important to discriminate between classes 0 and 1.

2.3 More than two classes

When the outcome Y refers to more than two classes, say m classes, then Fisher's method is constructed in exactly the same way. Now

$$\mathbf{W}^{-1}\mathbf{B}\mathbf{a} = \lambda\mathbf{a}$$

will have $r = \text{rank}(\mathbf{W}^{-1}\mathbf{B}) = \min(m-1, p, n)$ solutions (eigenvectors and eigenvalues). (n: sample size of training data)

Let $\mathbf{v}_j$ and λ_j denote the r solutions, and define

- $\mathbf{V}$: $p \times r$ matrix with columns $\mathbf{v}$
- $\mathbf{L}$: $r \times r$ diagonal matrix with elements $\lambda_1 > \lambda_2 > \cdots > \lambda_r$

The p-dimensional predictor data in $\mathbf{X}$ may then be transformed to the r-dimensional scores

$$\mathbf{Z} = \mathbf{X}\mathbf{V}.$$

For eigenvectors $\mathbf{v}_i$ and $\mathbf{v}_j$, it holds that

$$\text{cov}\left[Z_i, Z_j\right] = \text{cov}\left[\mathbf{X}\mathbf{v}_i, \mathbf{X}\mathbf{v}_j\right] = \mathbf{v}_i^t\mathbf{W}\mathbf{v}_j = \delta_{ij},$$

in which the covariances are defined within groups. Hence, within the groups (classes) the scores are uncorrelated.

3 High dimensional predictors

With high-dimensional predictors

- Replace the $p \times p$ matrices $\mathbf{W}$ and $\mathbf{B}$ by their diagonal matrices (i.e. put zeroes on the off-diagonal positions)
- **Sparse LDA** by imposing an L_1-penalty on $\mathbf{v}$.

Two approaches: Zhou *et al.* (2006), Journal of Computational and Graphical Statistics , **15**, 265-286, and Clemmensen *et al.* (2011), Technometrics, **53**.

4 Breast cancer example

4.1 All genes

4.1.1 LDA

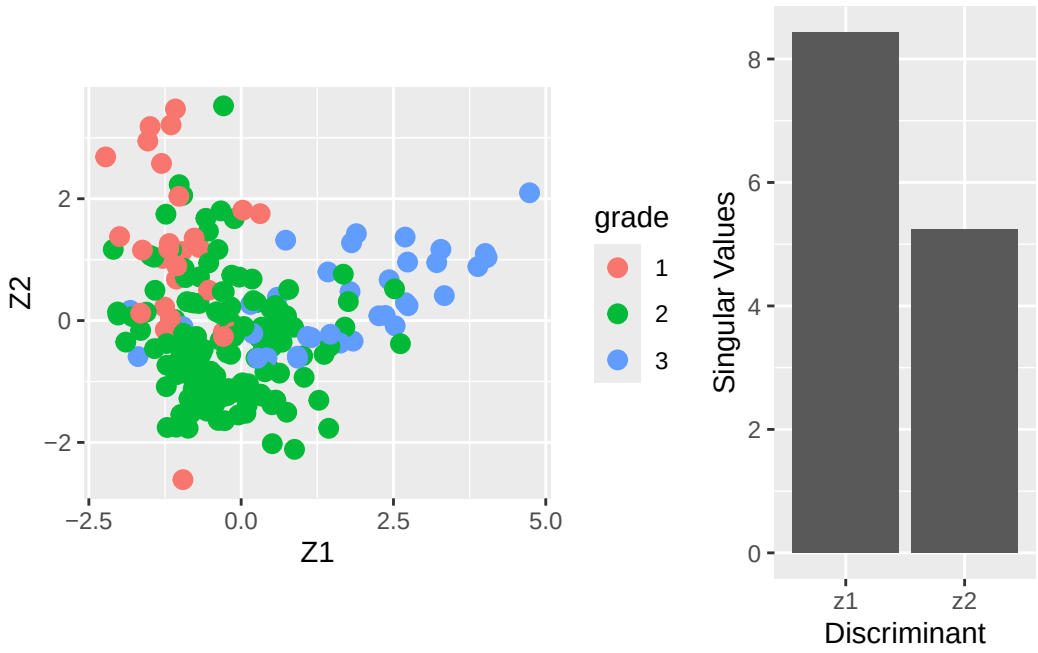
- Fisher's LDA is illustrated on the breast cancer data with all three tumor stages as outcome.
- We try to discriminate between the different stages according to the gene expression data of all genes.
- We cache the result because the calculation takes 10 minutes.

```
breast.lda <- MASS::lda(x = X, grouping = Y)
```

```
Vlda <- breast.lda$scaling
colnames(Vlda) <- paste0("V",1:ncol(Vlda))

Zlda <- X%*%Vlda
colnames(Zlda) <- paste0("Z",1:ncol(Zlda))

grid.arrange(
  Zlda |>
    as.data.frame() |>
    mutate(grade = Y |> as.factor()) |>
    ggplot(aes(x= Z1, y = Z2, color = grade)) +
    geom_point(size = 3) +
    coord_fixed(),
  ggplot() +
    geom_bar(aes(x = c("z1","z2"), y = breast.lda$svd), stat = "identity") +
    xlab("Discriminant") +
    ylab("Singular Values"),
  layout_matrix = matrix(
    c(1,1,2),
    nrow=1)
  )
```



- The columns of the matrix $\mathbf{V}$ contain the eigenvectors. There are $\min(3-2, 22283, 200) = 2$ eigenvectors. The 200×2 matrix $\mathbf{Z}$ contains the scores on the two Fisher discriminants.
- The eigenvalue λ_j can be interpreted as the ratio

$$\frac{\mathbf{v}_j^t \mathbf{B} \mathbf{v}_j}{\mathbf{v}_j^t \mathbf{W} \mathbf{v}_j},$$

  or (upon using $\mathbf{v}_j^t \mathbf{W} \mathbf{v}_j = 1$) the between-centroid sum of squares (in the reduced dimension space of the Fisher discriminants)

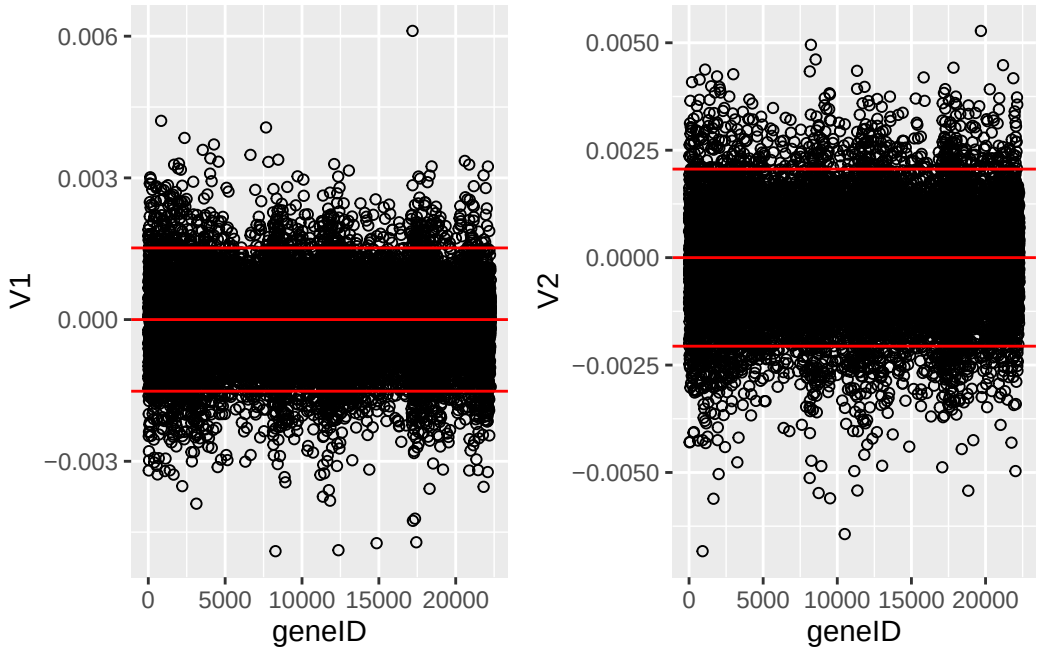
$$\mathbf{v}_j^t \mathbf{B} \mathbf{v}_j.$$

- From the screeplot of the eigenvalues we see that the first dimension is more important than the second (not hugely) in terms of discriminating between the groups.
- From the scatterplot we can see that there is no perfect separation (discrimination) between the three tumor stages (quite some overlap).
- To some extent the first Fisher discriminant dimension discriminates stage 3 (blue dots) from the other two stages, and the second dimension separates stage 1 (red dots) from the two others.

4.1.2 Interpretation of loadings

```
grid.arrange(
  Vlda |>
    as.data.frame() |>
    mutate(geneID = 1:nrow(Vlda)) |>
    ggplot(aes(x = geneID, y = V1)) +
    geom_point(pch=21) +
    geom_hline(yintercept = c(-2,0,2)*sd(Vlda[,1]), col = "red"),

  Vlda |>
    as.data.frame() |>
    mutate(geneID = 1:nrow(Vlda)) |>
    ggplot(aes(x = geneID, y = V2)) +
    geom_point(pch=21) +
    geom_hline(yintercept = c(-2,0,2)*sd(Vlda[,2]), col = "red"),
  ncol = 2)
```



The loadings of the Fisher discriminants are within the columns of the $\mathbf{V}$ matrix.

- Since we have 22283 genes, each discriminant is a linear combination of 22283 gene expression. Instead of looking at the listing of 22283 loadings, we made an index plot (no particular ordering of genes on horizontal axis).

- The red horizontal reference lines correspond to the average of the loading (close to zero) and the average plus and minus twice the standard deviation of the loadings.
- If no genes had any "significant" discriminating power, then we would expect approximately 95% of all loadings within the band. Thus loadings outside of the band are of potential interest and may perhaps be discriminating between the three tumor stages.
- In the graphs presented here we see many loadings within the bands, but also many outside of the band.

We repeat the analysis, but now with the sparse LDA method of Clemmensen et al. (2011).

4.2 Sparse LDA based on 150 random genes

- We only present the results of the sparse LDA based on a random subset of 150 genes (ordering of genes in datamatrix is random).
- The discrimination seems better than with classical LDA based on all genes. This is very likely caused by too much noise in the full data matrix with over 20000 predictors.

```
# BiocManager::install("sparseLDA")

library(sparseLDA)

YDummy <- data.frame(
  Y1 = ifelse(Y == 1, 1, 0),
  Y2 = ifelse(Y == 2, 1, 0),
  Y3 = ifelse(Y == 3, 1, 0)
  )

X2 <- X[,1:150]

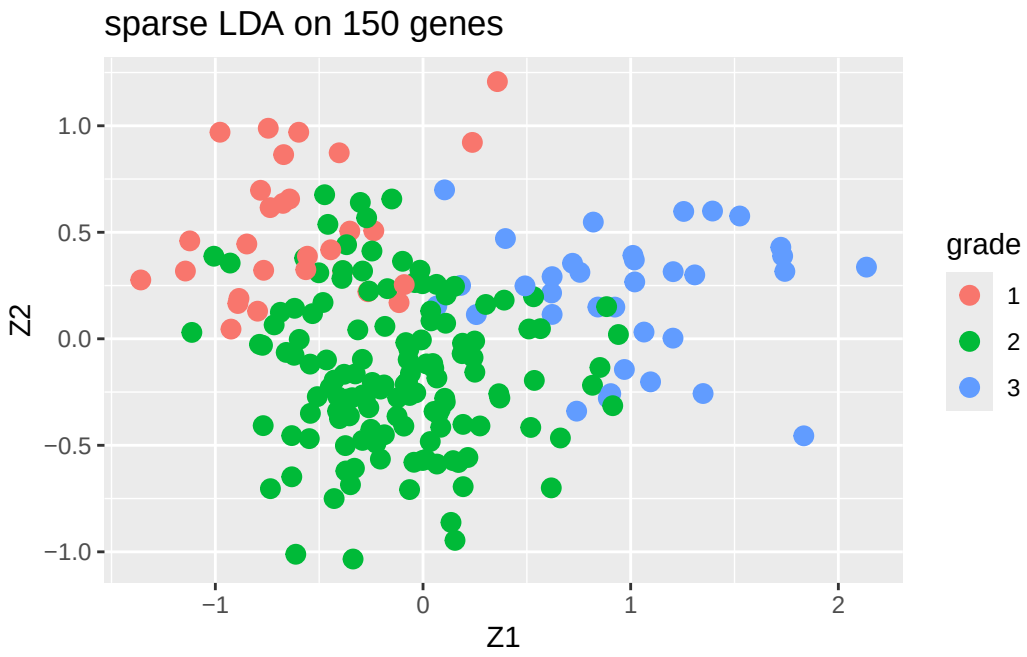
breast.slda <- sda(x = X2,
  y = as.matrix(YDummy),
  lambda = 1e-6,
  stop = -50,
  maxIte = 25,
  trace = TRUE)
```

```
#> ite:  1  ridge cost:  114.2886  |b|_1:  5.660842
#> ite:  2  ridge cost:  99.97973  |b|_1:  5.766274
#> ite:  3  ridge cost:  93.36374  |b|_1:  5.49319
#> ite:  4  ridge cost:  90.85553  |b|_1:  5.521626
#> ite:  5  ridge cost:  88.40883  |b|_1:  5.842209
#> ite:  6  ridge cost:  87.81743  |b|_1:  5.911008
#> ite:  7  ridge cost:  87.95633  |b|_1:  5.872692
#> ite:  8  ridge cost:  87.39725  |b|_1:  5.975554
#> ite:  9  ridge cost:  87.75908  |b|_1:  5.905802
#> ite:  10  ridge cost:  87.8732  |b|_1:  5.884001
#> ite:  11  ridge cost:  87.80685  |b|_1:  5.896474
#> ite:  12  ridge cost:  87.77241  |b|_1:  5.902971
#> ite:  13  ridge cost:  87.77107  |b|_1:  5.90322
#> ite:  14  ridge cost:  87.77644  |b|_1:  5.902204
#> ite:  15  ridge cost:  87.77909  |b|_1:  5.901702
#> ite:  16  ridge cost:  87.78039  |b|_1:  5.901455
#> ite:  17  ridge cost:  87.78104  |b|_1:  5.901334
#> ite:  18  ridge cost:  87.78135  |b|_1:  5.901274
#> ite:  19  ridge cost:  87.78151  |b|_1:  5.901244
#> ite:  20  ridge cost:  87.78159  |b|_1:  5.901229
#> ite:  1  ridge cost:  134.7777  |b|_1:  5.379261
#> ite:  2  ridge cost:  134.7777  |b|_1:  5.379261
#> final update, total ridge cost:  222.5593  |b|_1:  11.28049
```

```
Vsda <- matrix(0, nrow=ncol(X2), ncol=2)
Vsda[breast.slda$varIndex,] <- breast.slda$beta
colnames(Vsda) <- paste0("V",1:ncol(Vsda))

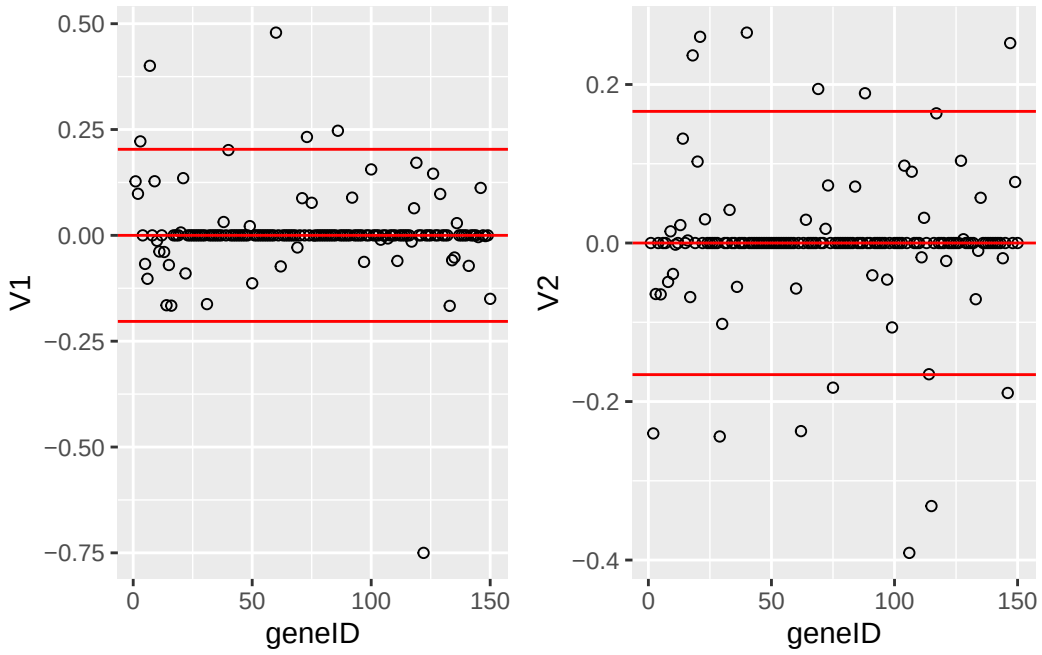
Zsda <- X2%*%Vsda
colnames(Zsda) <- paste0("Z",1:ncol(Zsda))

Zsda |>
 as.data.frame() |>
 mutate(grade = Y |> as.factor()) |>
 ggplot(aes(x= Z1, y = Z2, color = grade)) +
 geom_point(size = 3) +
 ggtitle("sparse LDA on 150 genes")
```




```
grid.arrange(
   Vsda |>
     as.data.frame() |>
     mutate(geneID = 1:nrow(Vsda)) |>
     ggplot(aes(x = geneID, y = V1)) +
     geom_point(pch=21) +
     geom_hline(yintercept = c(-2,0,2)*sd(Vsda[,1]), col = "red") ,

   Vsda |>
     as.data.frame() |>
     mutate(geneID = 1:nrow(Vsda)) |>
     ggplot(aes(x = geneID, y = V2)) +
     geom_point(pch=21) +
     geom_hline(yintercept = c(-2,0,2)*sd(Vsda[,2]), col = "red"),
   ncol = 2)
```



4.3 LDA based on 150 random genes

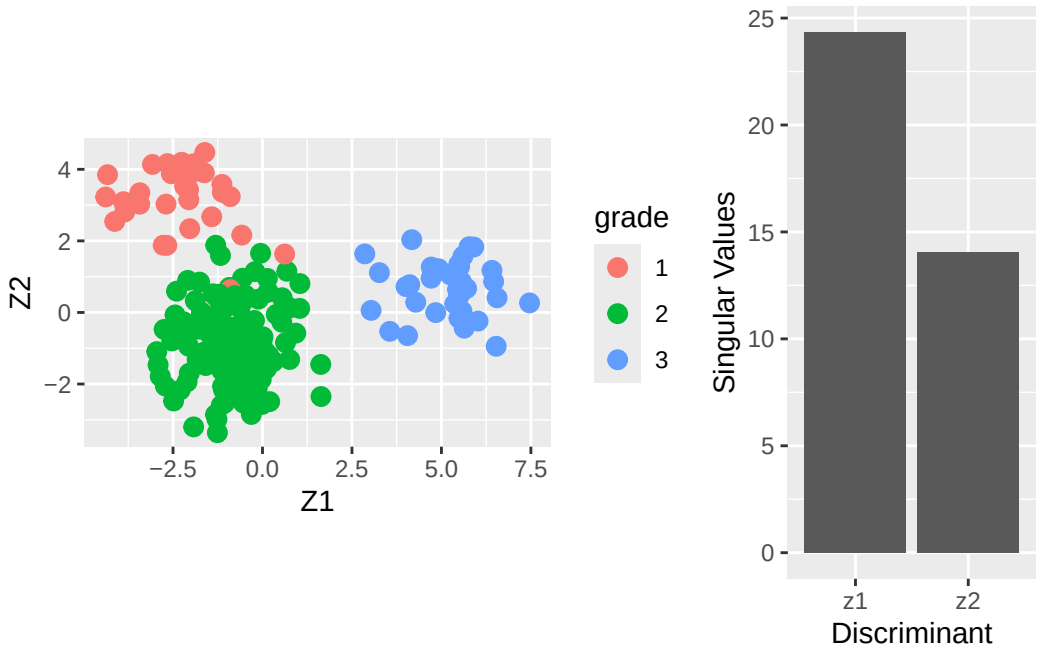
```
breast.lda150 <- MASS::lda(x = X2, grouping = Y)

Vlda <- breast.lda150$scaling
colnames(Vlda) <- paste0("V",1:ncol(Vlda))

Zlda <- X2%*%Vlda
colnames(Zlda) <- paste0("Z",1:ncol(Zlda))

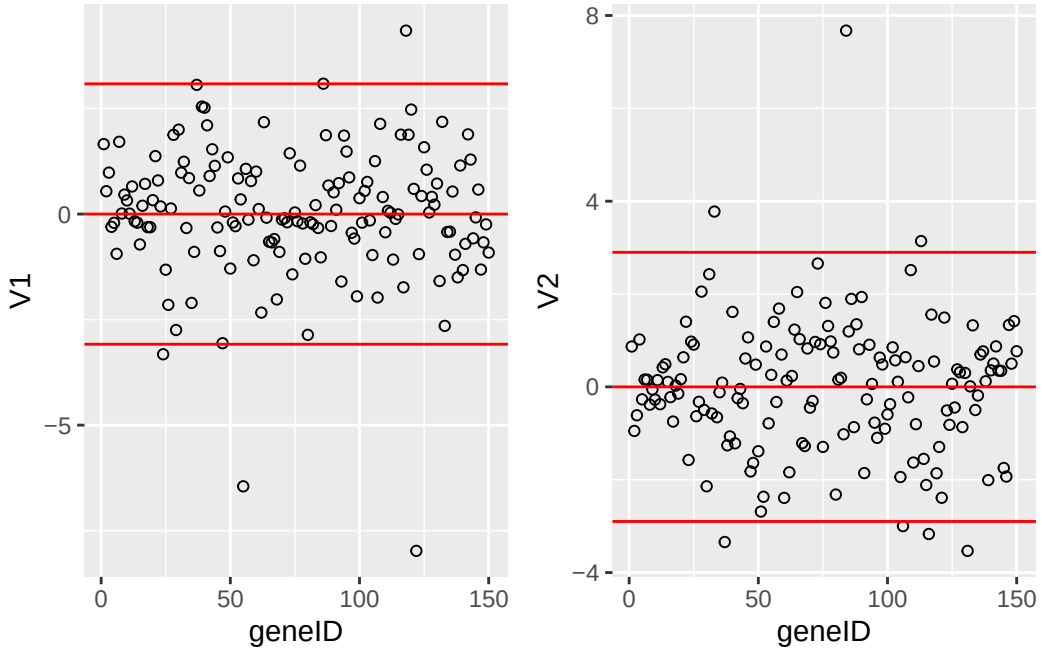
grid.arrange(
  Zlda |>
    as.data.frame() |>
    mutate(grade = Y |> as.factor()) |>
    ggplot(aes(x= Z1, y = Z2, color = grade)) +
    geom_point(size = 3) +
    coord_fixed(),
  ggplot() +
    geom_bar(aes(x = c("z1","z2"), y = breast.lda150$svd), stat = "identity") +
    xlab("Discriminant") +
    ylab("Singular Values"),
```

```
layout_matrix = matrix(
  c(1,1,2),
  nrow=1)
)
```




```
grid.arrange(
  Vlda |>
    as.data.frame() |>
    mutate(geneID = 1:nrow(Vlda)) |>
    ggplot(aes(x = geneID, y = V1)) +
    geom_point(pch=21) +
    geom_hline(yintercept = c(-2,0,2)*sd(Vlda[,1]), col = "red"),

  Vlda |>
    as.data.frame() |>
    mutate(geneID = 1:nrow(Vlda)) |>
    ggplot(aes(x = geneID, y = V2)) +
    geom_point(pch=21) +
    geom_hline(yintercept = c(-2,0,2)*sd(Vlda[,2]), col = "red"),
  ncol = 2)
```



4.4 Wrapup

LDA on all 22283 genes gave poorer result than on 150 genes. This is probably caused by numerical instability when working with large $\mathbf{W}$ and $\mathbf{B}$ matrices

- Sparse LDA gave slightly poorer result than LDA on the subset of 150 genes. This may be caused by overfitting of the LDA.
- When (sparse) LDA is used to build a prediction model/classifier, then CV methods, or splitting of dataset into training and test datasets should be used to allow for an honest evaluation of the final prediction model.
- The graphs in the first two Fisher discriminant dimensions shown on the previous slides should only be used for data exploration.
- When the objective is to try to understand differences between groups in a high dimensional space, Fisher LDA is preferred over PCA.

Acknowledgement

- Olivier Thas for sharing his materials of Analysis of High Dimensional Data 2019-2020, which I used as the starting point for this chapter.

Session info

Session info

```
#> [1] "2026-10-22 13:32:31 CEST"
```

```
#> - Session info ---------------------------------------------------------------
#>  setting  value
#>  version  R version 4.6.1 (2026-06-24)
#>  os       macOS Tahoe 26.1
#>  system   aarch64, darwin23
#>  ui       X11
#>  language (EN)
#>  collate  en_US.UTF-8
#>  ctype    en_US.UTF-8
#>  tz       Europe/Brussels
#>  date     2026-10-22
#>  pandoc   3.10.2 @ /opt/homebrew/bin/ (via rmarkdown)
#>  quarto   1.9.37 @ /usr/local/bin/quarto
#>
#> - Packages -------------------------------------------------------------------
#>  package           * version    date (UTC) lib source
#>  AIMS              * 1.44.0     2026-04-28 [1] Bioconductor 3.23 (R 4.6.0)
#>  AnnotationDbi       1.74.0     2026-04-28 [1] Bioconductor 3.23 (R 4.6.0)
#>  Biobase           * 2.72.0     2026-04-28 [1] Bioconductor 3.23 (R 4.6.0)
#>  BiocFileCache       3.2.0      2026-04-28 [1] Bioconductor 3.23 (R 4.6.0)
#>  BiocGenerics      * 0.58.1     2026-05-14 [1] https://bioc-release.r-universe.dev (R 4.6
#>  biomaRt           * 2.68.0     2026-04-28 [1] Bioconductor 3.23 (R 4.6.0)
#>  Biostrings          2.80.1     2026-05-22 [1] https://bioc-release.r-universe.dev (R 4.6
#>  bit                 4.6.0      2025-03-06 [1] CRAN (R 4.6.0)
#>  bit64               4.8.2      2026-05-19 [1] CRAN (R 4.6.0)
#>  blob                1.3.0      2026-01-14 [1] CRAN (R 4.6.0)
#>  bootstrap           2019.6     2019-06-17 [1] CRAN (R 4.6.0)
#>  breastCancerMAINZ * 1.50.0     2026-05-05 [1] Bioconductor 3.23 (R 4.6.1)
#>  cachem              1.1.0      2024-05-16 [1] CRAN (R 4.6.0)
#>  class               7.3-23     2025-01-01 [1] CRAN (R 4.6.1)
#>  cli                 3.6.6      2026-04-09 [1] CRAN (R 4.6.0)
#>  cluster             2.1.8.2    2026-02-05 [1] CRAN (R 4.6.1)
#>  codetools           0.2-20     2024-03-31 [1] CRAN (R 4.6.1)
#>  crayon              1.5.3      2024-06-20 [1] CRAN (R 4.6.0)
#>  curl                7.1.0      2026-04-22 [1] CRAN (R 4.6.0)
#>  data.table          1.18.4     2026-05-06 [1] CRAN (R 4.6.0)
```

```
#>  DBI                 1.3.0      2026-02-25 [1] CRAN (R 4.6.0)
#>  dbplyr              2.6.0      2026-06-17 [1] CRAN (R 4.6.0)
#>  digest              0.6.39     2025-11-19 [1] CRAN (R 4.6.0)
#>  dplyr             * 1.2.1      2026-04-03 [1] CRAN (R 4.6.0)
#>  e1071             * 1.7-17     2025-12-18 [1] CRAN (R 4.6.0)
#>  elasticnet          1.3        2020-05-15 [1] CRAN (R 4.6.0)
#>  evaluate            1.0.5      2025-08-27 [1] CRAN (R 4.6.0)
#>  farver              2.1.2      2024-05-13 [1] CRAN (R 4.6.0)
#>  fastmap             1.2.0      2024-05-15 [1] CRAN (R 4.6.0)
#>  filelock            1.0.3      2023-12-11 [1] CRAN (R 4.6.0)
#>  forcats           * 1.0.1      2025-09-25 [1] CRAN (R 4.6.0)
#>  future              1.75.0     2026-07-20 [1] CRAN (R 4.6.1)
#>  future.apply        1.20.2     2026-02-20 [1] CRAN (R 4.6.0)
#>  genefu            * 2.44.0     2026-04-28 [1] Bioconductor 3.23 (R 4.6.0)
#>  generics          * 0.1.4      2025-05-09 [1] CRAN (R 4.6.0)
#>  ggplot2           * 4.0.3      2026-04-22 [1] CRAN (R 4.6.0)
#>  globals             0.19.1     2026-03-13 [1] CRAN (R 4.6.0)
#>  glue                1.8.1      2026-04-17 [1] CRAN (R 4.6.0)
#>  gridExtra         * 2.3.1      2026-06-25 [1] CRAN (R 4.6.1)
#>  gtable              0.3.6      2024-10-25 [1] CRAN (R 4.6.0)
#>  hms                 1.1.4      2025-10-17 [1] CRAN (R 4.6.0)
#>  htmltools           0.5.9      2025-12-04 [1] CRAN (R 4.6.0)
#>  httr                1.4.8      2026-02-13 [1] CRAN (R 4.6.0)
#>  httr2               1.2.3      2026-06-23 [1] CRAN (R 4.6.1)
#>  iC10              * 2.0.3      2026-08-04 [1] CRAN (R 4.6.1)
#>  iC10TrainingData    2.0.1      2024-07-16 [1] CRAN (R 4.6.0)
#>  impute              1.86.0     2026-04-28 [1] Bioconductor 3.23 (R 4.6.0)
#>  IRanges             2.46.0     2026-04-28 [1] Bioconductor 3.23 (R 4.6.0)
#>  jsonlite            2.0.0      2025-03-27 [1] CRAN (R 4.6.0)
#>  KEGGREST            1.52.0     2026-04-28 [1] Bioconductor 3.23 (R 4.6.0)
#>  KernSmooth          2.23-26    2025-01-01 [1] CRAN (R 4.6.1)
#>  knitr               1.51       2025-12-20 [1] CRAN (R 4.6.0)
#>  labeling            0.4.3      2023-08-29 [1] CRAN (R 4.6.0)
#>  lars                1.3        2022-04-13 [1] CRAN (R 4.6.0)
#>  lattice             0.22-9     2026-02-09 [1] CRAN (R 4.6.1)
#>  lava                1.9.3      2026-08-20 [1] CRAN (R 4.6.1)
#>  lifecycle           1.0.5      2026-01-08 [1] CRAN (R 4.6.0)
#>  limma               3.68.4     2026-05-31 [1] https://bioc-release.r-universe.dev (R 4.6
#>  listenv             1.0.0      2026-06-22 [1] CRAN (R 4.6.1)
#>  lubridate         * 1.9.5      2026-02-04 [1] CRAN (R 4.6.0)
#>  magrittr            2.0.5      2026-04-04 [1] CRAN (R 4.6.0)
#>  MASS                7.3-65     2025-02-28 [1] CRAN (R 4.6.1)
#>  Matrix              1.7-5      2026-03-21 [1] CRAN (R 4.6.1)
```

```
#>  mclust             6.1.3      2026-07-05 [1] CRAN (R 4.6.1)
#>  mda                0.5-5      2024-11-07 [1] CRAN (R 4.6.0)
#>  memoise            2.0.1      2021-11-26 [1] CRAN (R 4.6.0)
#>  otel               0.2.0      2025-08-29 [1] CRAN (R 4.6.0)
#>  pamr               1.57       2024-07-01 [1] CRAN (R 4.6.0)
#>  parallelly         1.48.0     2026-06-29 [1] CRAN (R 4.6.1)
#>  pillar             1.11.1     2025-09-17 [1] CRAN (R 4.6.0)
#>  pkgconfig          2.0.3      2019-09-22 [1] CRAN (R 4.6.0)
#>  png                0.1-9      2026-03-15 [1] CRAN (R 4.6.0)
#>  prettyunits        1.2.0      2023-09-24 [1] CRAN (R 4.6.0)
#>  prodlim          * 2026.03.11 2026-03-11 [1] CRAN (R 4.6.0)
#>  progress           1.2.3      2023-12-06 [1] CRAN (R 4.6.0)
#>  proxy              0.4-29     2025-12-29 [1] CRAN (R 4.6.0)
#>  purrr            * 1.2.2      2026-04-10 [1] CRAN (R 4.6.0)
#>  R6                 2.6.1      2025-02-15 [1] CRAN (R 4.6.0)
#>  rappdirs           0.3.4      2026-01-17 [1] CRAN (R 4.6.0)
#>  RColorBrewer       1.1-3      2022-04-03 [1] CRAN (R 4.6.0)
#>  Rcpp               1.1.1-1.1  2026-04-24 [1] CRAN (R 4.6.0)
#>  readr            * 2.2.0      2026-02-19 [1] CRAN (R 4.6.0)
#>  rlang              1.2.0      2026-04-06 [1] CRAN (R 4.6.0)
#>  rmarkdown          2.31       2026-03-26 [1] CRAN (R 4.6.0)
#>  rmeta              3.0        2018-03-20 [1] CRAN (R 4.6.0)
#>  RSQLite            3.53.2     2026-06-17 [1] CRAN (R 4.6.0)
#>  rstudioapi         0.19.0     2026-06-11 [1] CRAN (R 4.6.0)
#>  S4Vectors          0.50.1     2026-05-03 [1] https://bioc-release.r-universe.dev (R 4.6
#>  S7                 0.2.2      2026-04-22 [1] CRAN (R 4.6.0)
#>  scales             1.4.0      2025-04-24 [1] CRAN (R 4.6.0)
#>  Seqinfo            1.2.0      2026-04-28 [1] Bioconductor 3.23 (R 4.6.0)
#>  sessioninfo        1.2.4      2026-06-04 [1] CRAN (R 4.6.0)
#>  sparseLDA        * 0.1-9      2016-09-22 [1] CRAN (R 4.6.0)
#>  statmod            1.5.2      2026-05-17 [1] CRAN (R 4.6.0)
#>  stringi            1.8.7      2025-03-27 [1] CRAN (R 4.6.0)
#>  stringr          * 1.6.0      2025-11-04 [1] CRAN (R 4.6.0)
#>  SuppDists          1.1-9.9    2025-03-24 [1] CRAN (R 4.6.0)
#>  survcomp         * 1.62.0     2026-04-28 [1] Bioconductor 3.23 (R 4.6.0)
#>  survival         * 3.8-6      2026-01-16 [1] CRAN (R 4.6.1)
#>  survivalROC        1.0.3.1    2022-12-05 [1] CRAN (R 4.6.0)
#>  tibble           * 3.3.1      2026-01-11 [1] CRAN (R 4.6.0)
#>  tidyr            * 1.3.2      2025-12-19 [1] CRAN (R 4.6.0)
#>  tidyselect         1.2.1      2024-03-11 [1] CRAN (R 4.6.0)
#>  tidyverse        * 2.0.0      2023-02-22 [1] CRAN (R 4.6.0)
#>  timechange         0.4.0      2026-01-29 [1] CRAN (R 4.6.0)
#>  tzdb               0.5.0      2025-03-15 [1] CRAN (R 4.6.0)
```

```
#>  vctrs               0.7.3      2026-04-11 [1] CRAN (R 4.6.0)
#>  withr               3.0.3      2026-06-19 [1] CRAN (R 4.6.0)
#>  xfun                0.59       2026-06-19 [1] CRAN (R 4.6.0)
#>  XVector             0.52.0     2026-04-28 [1] Bioconductor 3.23 (R 4.6.0)
#>  yaml                2.3.12     2025-12-10 [1] CRAN (R 4.6.0)
#>
#>  [1] /Library/Frameworks/R.framework/Versions/4.6/Resources/library
#>  * -- Packages attached to the search path.
#>
#> ------------------------------------------------------------------------------
```